

Meridian data engineering assessment

Candidate assignment | 1-2 day take-home | 60-minute review

Your assignment

Meridian Service Cooperative operates service centres under three acquired businesses: Aster Field Services, Birch Care Operations and Cobalt Support Network. Each business runs its own case system behind a separately managed private connection. Leadership wants one reliable operational view without replacing those systems.

Design and implement a data pipeline that delivers this view on one small production server. You own the design, implementation and explanation of its tradeoffs. We will review how you discover missing requirements, handle unreliable connections, keep data correct, allocate resources and protect access.

This is a senior take-home assessment with a 1-2 day window and approximately 10-14 focused hours of work, followed by a 60-minute review. Tell us what you prioritized and what you would do next. The supplied technical contract defines the testable interfaces. It does not prescribe a database, orchestrator, message broker or deployment architecture.

What the business asks for

Regional managers want daily numbers for cases and service value across their centres. They need to distinguish completed, open and cancelled work, and they need to trust the numbers when a vendor corrects old records. Today, two people can export the same report and get different answers.

The operations team wants to know whether a source is current or disconnected. One business's connection often fails while the others remain available. Some vendor applications use the same internal hostname, but the connection owners will not change their networks to accommodate us.

Managers should only see information relevant to their work. The assurance team sometimes needs to inspect a case and trace where it came from. Contact details and sensitive cases need tighter handling than routine summary information. An operator who restarts jobs should not automatically gain access to business data.

The cooperative expects another acquisition and new reporting questions. We want changes to be manageable without rewriting the whole pipeline. Explain how your design could accommodate a fourth source or a changed field definition, and where you would deliberately stop generalizing.

What you should clarify

Ask questions before you finalize the design. Record your assumptions, why they matter and how a different answer would affect implementation. We can answer questions about business meaning, freshness, access, history, recovery and likely changes. We care about the quality and prioritization of your questions, not the number asked.

The interviewer will provide a short clarification sheet before implementation is scored, and may introduce a bounded change during review. Do not assume every unspecified feature is required. State what is implemented and what remains a proposal. No UI, cloud deployment, enterprise identity provider or actual VPN tunnel is required.

What you must deliver

Provide a running containerized pipeline that consumes all three mock APIs, creates a queryable operational view and exposes the result endpoints in the technical contract. Make a repeat run and an application restart safe. Show how failures and bad input become visible and how the service recovers after connectivity returns.

Enforce access to the business view and case details using the supplied identity fixture. Explain the trust boundaries, access rules, secret handling, data classification, retention approach and audit evidence. Include tests for both permitted and denied access.

Include a concise architecture diagram and decision log. Explain your ingestion strategy, storage model, update semantics, deployment units, failure handling, observability, migration approach and resource allocation. Compare at least one plausible alternative and explain why you did not choose it for this server.

Provide a runbook for setup, operation, recovery, credential rotation and adding a source. Include a resource and performance report with reproducible commands, measured hardware, dataset size and known limits. We value evidence and clear constraints more than large throughput claims.

Fixed operating conditions

Choose and document suitable CPU and memory allocations for your application, including its database, worker, API, scheduler and any sidecars. Set service limits appropriate to your implementation and explain your resource decisions. Keep persistent application data within 2 GiB. Account for connection pools, temporary space and background processes.

Use the provided Docker networks and authenticated TLS gateway endpoints. The infrastructure is immutable. Do not bypass gateways, disable certificate validation, connect to private source networks, use host networking, mount the Docker socket, run privileged containers or ingest fixture files directly. The technical contract explains the allowed attachment points.

Use approximately 10,000 supplied synthetic events. Expect differences between source representations and changes to existing cases. Source IDs need not be globally unique. Runtime ingestion must use the APIs, even though the backing fixtures are available for inspection.

How the review works

We will first ask you to walk through your design and assumptions, then run your submission with fresh credentials. We will examine correctness, repeatability, recovery, network isolation, resource use and authorization. You will investigate a source interruption and discuss a small requirement change.